

« - »

« №1
»
bf16

5130201/40002

_____.
_____.

«____» _____ 2026 .

-
2026

1

1. , :

2. bf16. ,

2

bf16 -
M.
16 : 1 s, 8 E (127) 7
1.M · 2^{E-127}, . :
x = (-1)^s · (1 + M) · 2^{E-127}.
: E = 0 M = 0 , - x = (-1)^s · M · 2¹⁻¹²⁷;
E = 255 ±∞ (M = 0) NaN (M ≠ 0).
bf16.
2, - 2 (
1.xxxxx · 2^E , 7 ,)
127. (), E
bf16 . : 16 , NaN).
(, ,
() - 2 ()
().

3

	Python	: main.py	,
bf16.py -		bf16.	bf16.py.


```

decimal_to_bf16
: float value -
:      16      -      bf16
:
fractional_part_to_binary).
7 (round_mantissa),
(      0).
```

```

(unsigned_integer_to_binary,
1.mantissa · 2exponent (normalize),
127. (      ±∞)
```

```

1 def decimal_to_bf16(value: float) -> str:
2     if value == 0:
3         return "0" * TOTAL_BITS
4
5     sign_bit = "1" if value < 0 else "0"
6     module = abs(value)
7
8     integer_part = int(module)
9     fractional_part = module - integer_part
10
11     integer_bits = unsigned_integer_to_binary(integer_part)
12     fractional_bits = fractional_part_to_binary(fractional_part)
13     exponent, mantissa_source = normalize(integer_bits, fractional_bits)
14     if exponent is None:
15         return sign_bit + "0" * (EXPONENT_BITS + MANTISSA_BITS)
16
17     mantissa_bits, exponent_bump = round_mantissa(mantissa_source, MANTISSA_BITS)
18     biased_exponent = exponent + exponent_bump + BIAS
19
20     if biased_exponent >= 255:
21         return sign_bit + "1" * EXPONENT_BITS + "0" * MANTISSA_BITS
22     if biased_exponent <= 0:
23         return sign_bit + "0" * (EXPONENT_BITS + MANTISSA_BITS)
24
25     exponent_bits = format(biased_exponent, f"0{EXPONENT_BITS}b")
26     return sign_bit + exponent_bits + mantissa_bits
```

1: decimal_to_bf16

```

bf16_to_decimal
: str bits -      16      bf16
: float -
:
:      ,      .
:      0      255
±∞ NaN,      (-1)sign(1 + mantissa) · 2exponent-127.
```

```

(bits_to_fraction),
```

```

1 def bf16_to_decimal(bits: str) -> float:
2     bits = bits.strip()
3     if len(bits) != TOTAL_BITS or any(bit not in "01" for bit in bits):
4         raise ValueError(f"    {TOTAL_BITS} ,    0_1")
5
6     sign = -1 if bits[0] == "1" else 1
7     exponent_bits = bits[1:1 + EXPONENT_BITS]
8     mantissa_bits = bits[1 + EXPONENT_BITS:]
9     biased_exponent = int(exponent_bits, 2)
10    mantissa_fraction = bits_to_fraction(mantissa_bits)
11
```

```

12     if biased_exponent == 0:
13         if mantissa_fraction == 0:
14             return sign * 0.0
15         return sign * mantissa_fraction * 2 ** (1 - BIAS)
16
17     if biased_exponent == 255:
18         return sign * float("inf") if mantissa_fraction == 0 else float("nan")
19
20     return sign * (1 + mantissa_fraction) * 2 ** (biased_exponent - BIAS)

```

2: bf16_to_decimal

```

describe_bf16
: str bits -      16      bf16
:
:      ,
:      :      16      ,
:      -      .

```

```

1 def describe_bf16(bits: str) -> str:
2     sign_bit = bits[0]
3     exponent_bits = bits[1:1 + EXPONENT_BITS]
4     mantissa_bits = bits[1 + EXPONENT_BITS:]
5     biased_exponent = int(exponent_bits, 2)
6     return (
7         f"    _={sign_bit}_    (('{_if sign_bit==_1'_else_    _'})\n"
8         f"    _={exponent_bits}_->_    _{biased_exponent},_"
9         f"    _{biased_exponent}_-_{BIAS}_    (_-{BIAS})\n"
10        f"    _={mantissa_bits}_->_    _    _{bits_to_fraction(mantissa_bits)}"
11    )

```

3: describe_bf16

```

unsigned_integer_to_binary
: int value -
: str -
:
:      2:      2 -
(      ),      2.

```

```

1 def unsigned_integer_to_binary(value: int) -> str:
2     if value == 0:
3         return "0"
4     binary_text = ""
5     while value > 0:
6         binary_text = str(value % 2) + binary_text
7         value //= 2
8     return binary_text

```

4: unsigned_integer_to_binary

```

fractional_part_to_binary
: float fractional_part -      ,  $0 \leq x < 1$ 
: str -
:
:      2:      2,      -
,      ,      ,      (      float -      -
,      ,      ),      MAX_USEFUL_FRACTIONAL_BITS
-      ,      bf16 -      .

```

```

1 def fractional_part_to_binary(fractional_part: float) -> str:
2     bits = []
3     for _ in range(MAX_USEFUL_FRACTIONAL_BITS):
4         if fractional_part == 0:

```

```
5         break
6         fractional_part *= 2
7         bit = int(fractional_part)
8         bits.append(str(bit))
9         fractional_part -= bit
10    return "".join(bits)
```

5: fractional_part_to_binary

```

normalize
: str integer_bits, fractional_bits -
: exponent - ( None, bf16); mantissa_source -
: , ( ),
. , - ,
- : , bf16.

```

```

1 def normalize(integer_bits: str, fractional_bits: str):
2     if integer_bits != "0":
3         exponent = len(integer_bits) - 1
4         mantissa_source = integer_bits[1:] + fractional_bits
5         return exponent, mantissa_source
6
7     first_one_index = fractional_bits.find("1")
8     if first_one_index == -1:
9         return None, ""
10    exponent = -(first_one_index + 1)
11    mantissa_source = fractional_bits[first_one_index + 1:]
12    return exponent, mantissa_source

```

6: normalize

$$\begin{aligned} & \text{round_mantissa} \\ & : \text{str bits - } ; \text{int keep - } (7 \quad \text{bf16}) \\ & : ; (0 \quad 1) \\ & : \text{keep , } : 0, \\ & , 1 - . (111 \dots 1), \\ & , , 1. \end{aligned}$$

```

1 def round_mantissa(bits: str, keep: int):
2     bits = bits.ljust(keep + 1, "0")
3     kept_bits, round_bit = bits[:keep], bits[keep]
4     if round_bit == "0":
5         return kept_bits, 0
6
7     incremented = bin(int(kept_bits, 2) + 1)[2:].zfill(keep)
8     if len(incremented) > keep:
9         return "0" * keep, 1
10    return incremented, 0

```

7: round mantissa

$$\sum_i bit_i \cdot 2^{-(i+1)},$$

```
1 def bits_to_fraction(bits: str) -> float:
2     return sum(int(bit) * 2 ** -(position + 1) for position, bit in enumerate(bits))
```

8: bits_to_fraction

4

bf16 7 , -123.45 bf16 , 9. (-123.5 -123.45).

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30

```

9: -123.45 bf16

bf16

().

:

1. 7 , , bf16 : , , , NaN.
2. ($\leftrightarrow$) (/).